



SCHOOL *of* DATA SCIENCE

# MLlib Method Summary and Implementation Details

---

Distributed Computing  
School of Data Science  
University of Virginia

Last updated: September 24, 2025

# MLlib DataFrames API

The interface makes it relatively easy to experiment across models

The common activities include:

- **fit()** to train a model
- **transform()** to make predictions
- **evaluate()** to assess predictive performance

# MLlib DataFrames API, contd.

Methods are analogous for different tasks e.g., regression, classification

Will call different library, different functions but ideas are the same

## Regression

```
from pyspark.ml.regression import LinearRegression

lr = LinearRegression(featuresCol='features',
                      labelCol='median_house_value',
                      maxIter=10,
                      regParam=0.3,
                      elasticNetParam=0.8)

# Fit the model
lrModel = lr.fit(tr)
```

## Classification

```
from pyspark.ml.classification import LogisticRegression

# instantiate the model
lr = LogisticRegression(labelCol='high_price',
                       featuresCol='scaledFeatures',
                       maxIter=10,
                       regParam=0.3,
                       elasticNetParam=0.8)

# Fit the model
lrModel = lr.fit(scaledData)
```

# Train a Model

Import the model from the **ml** library

Instantiate the model by passing:

- name of features column
- name of label column
- hyperparameters

```
from pyspark.ml.regression import LinearRegression

lr = LinearRegression(featuresCol='features',          # feature vector name
                      labelCol='median_house_value',  # target variable name
                      maxIter=10,
                      regParam=0.3,
                      elasticNetParam=0.8)

# Fit the model
lrModel = lr.fit(tr)
```

# Make Predictions

Predict with the **transform()** method

```
# compute predictions. this will append column "prediction" to dataframe  
lrPred = lrModel.transform(tr)
```

Inspect a record with some columns

```
lrPred.select('median_house_value', 'median_income', 'features', 'prediction').show(1)
```

```
+-----+-----+-----+-----+  
|median_house_value|median_income|    features|    prediction|  
+-----+-----+-----+-----+  
|          452600.0|         8.3252|[8.3252,880.0]|417764.70239237114|  
+-----+-----+-----+-----+  
only showing top 1 row
```

# Assess Predictive Performance

Import an evaluator and call **evaluate()** method

Pass name of prediction column, label column

```
from pyspark.ml.evaluation import RegressionEvaluator

ev = RegressionEvaluator(predictionCol="prediction", labelCol="median_house_value")

print('-'*20)
print("METRICS")
print("Mean Squared Error:", ev.evaluate(lrPred, {ev.metricName: "mse"}))
print("R Squared:", ev.evaluate(lrPred, {ev.metricName: 'r2'}))
```

```
-----
METRICS
Mean Squared Error: 703796169.4197685
R Squared: 0.6032614088503241
```

# Spark MLlib Implementation Details

Source: <https://stanford.edu/~rezab/slides/strata2015.pdf>

# Matrix Operations are Foundational

How to distribute a matrix across machines?

- » By Entries (CoordinateMatrix)
- » By Rows (RowMatrix)
- » By Blocks (BlockMatrix)      As of version 1.3

# Mllib Data Handling

- > Data stored in distributed collections partitioned across nodes in cluster
- > Model training algorithms implemented to **aggregate statistics from partitions** and efficiently compute parameters

# Mllib Data Handling

- > Data stored in distributed collections partitioned across nodes in cluster
- > Model training algorithms implemented to **aggregate statistics from partitions** and efficiently compute parameters

## **Examples:**

- > *Linear models*: use distributed gradient descent
  - Each worker computes partial gradients on its partition
  - Partial gradients are aggregated
- > *Tree-based models*: feature statistics (e.g., node impurity) computed in distributed manner

# Logistic Regression

$$w \leftarrow w - \alpha \cdot \sum_{i=1}^n g(w; x_i, y_i)$$

```
val points = spark.textFile(...).map(parsePoint).cache()
var w = Vector.zeros(d)
for (i <- 1 to numIterations) {
  val gradient = points.map { p =>
    (1 / (1 + exp(-p.y * w.dot(p.x)) - 1) * p.y * p.x
  ).reduce(_ + _)
  w -= alpha * gradient
}
```

Each iteration calls  
training data, so cache  
it for speedup

Aggregate partial gradients  
across partitions